

Numerical Project in Python n.1: Image Deblurring

4OPT2 - Continuous Optimisation

Nathanaël SEROPIAN & Ramzi JEBALI

February 2026

Introduction

The goal of this project is to solve the optimization problem $\min_x \frac{1}{2}\|Ax - b\|_2^2 + \epsilon\|x\|_1$, which aims at recovering an image x such that, once blurred by the operator A , it matches the observed blurred image b . To this end, we implement a forward-backward method with Nesterov's acceleration (FISTA) to address this ℓ_1 -regularized deblurring problem.

1 Theoretical Part

Question 1: Convexity and Smoothness

The objective function F is the sum of two functions:

- $f_1(x) = \frac{1}{2}\|Ax - b\|_2^2$. Its Hessian is $\nabla^2 f_1(x) = A^T A$ which is positive semi-definite. Thus, f_1 is convex.
- $f_2(x) = \epsilon\|x\|_1$. This is a norm (multiplied by $\epsilon > 0$). By definition, all norms are convex functions.

Since the sum of two convex functions is convex, F is convex. As we minimize a convex function over a convex set, the problem is convex.

$$0 \preceq \nabla^2 f_1(x) = A^T A \preceq \sigma_{\max}^2(A)$$

so $f_1 \in \mathcal{C}_L^{1,1}$. Thus, as f_1 is convex, $f_1 \in \mathcal{S}_{0,L}^{1,1}$ with the Lipschitz constant: $L = \sigma_{\max}^2(A) = \lambda_{\max}(A^T A)$

Question 2: ISTA Formulation

We want to solve $\min_x f_1(x) + f_2(x)$, with $f_1(x) = \frac{1}{2}\|Ax - b\|_2^2$ and $f_2(x) = \epsilon\|x\|_1$. The proximal gradient method reads:

$$x_{k+1} = \text{prox}_{\alpha f_2}(x_k - \alpha \nabla f_1(x_k)) = \arg \min_{z \in \mathbb{R}^n} \left(\alpha \epsilon \|z\|_1 + \frac{1}{2} \|z - (x_k - \alpha \nabla f_1(x_k))\|_2^2 \right). \quad (1)$$

First, we compute the gradient of the smooth part: $\nabla f_1(x_k) = A^T(Ax_k - b)$.

Solving the proximal step calling $v_k = x_k - \alpha \nabla f_1(x_k)$, the problem becomes:

$$\arg \min_{z \in \mathbb{R}^n} \left(\alpha \epsilon \|z\|_1 + \frac{1}{2} \|z - v_k\|_2^2 \right) \implies \alpha \epsilon \partial \|z\|_1 + (z - v_k) \ni 0. \quad (2)$$

The last set of equations reads, for all components i : $\alpha \epsilon \partial |[z]_i| + [z]_i \ni [v_k]_i$.

We have now several possibilities, recalling that $\partial |[z]_i|$ depends on the sign of $[z]_i$:

$$[z]_i > 0 \implies [z]_i = [v_k]_i - \alpha \epsilon \quad (\text{which is valid then for } [v_k]_i > \alpha \epsilon) \quad (3)$$

$$[z]_i < 0 \implies [z]_i = [v_k]_i + \alpha \epsilon \quad (\text{which is valid then for } [v_k]_i < -\alpha \epsilon) \quad (4)$$

$$[z]_i = 0 \implies \partial |[z]_i| \in [-1, 1] \text{ and } [v_k]_i \in [-\alpha \epsilon, \alpha \epsilon]. \quad (5)$$

Reversing, we find the next iterate $[x_{k+1}]_i = [z^*]_i$:

$$[x_{k+1}]_i = \begin{cases} [v_k]_i - \alpha \epsilon & \text{for } [v_k]_i > \alpha \epsilon \\ [v_k]_i + \alpha \epsilon & \text{for } [v_k]_i < -\alpha \epsilon \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

and compactifying using the positive part operator $(\cdot)_+ = \max(\cdot, 0)$:

$$[x_{k+1}]_i = \text{sign}([v_k]_i) (|[v_k]_i| - \alpha \epsilon)_+. \quad (7)$$

This demonstrates that the proximal gradient applied to the ℓ_1 regularized problem yields the exact Iterative Soft-Thresholding Algorithm (ISTA) iterations provided in Equation (1).

Question 3: Implementation and Convergence

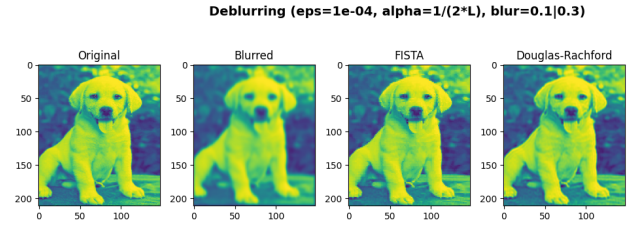
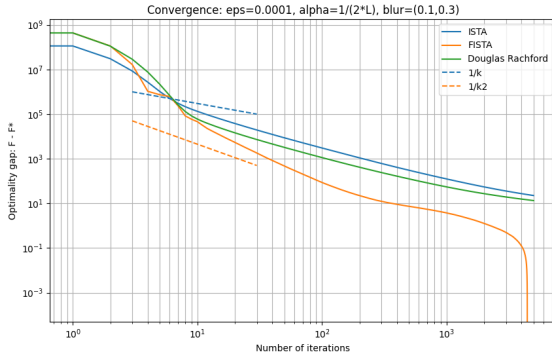


Figure 1: **Base Configuration** ($\epsilon = 10^{-4}$, $p = 2$, **weak blur**): FISTA and Douglas-Rachford converge significantly faster than ISTA. The image is successfully restored with well-preserved edges.

We implemented both ISTA and FISTA in Python. To optimize performance, the soft-thresholding operator was fully vectorized using NumPy functions, completely avoiding for loops. As theoretically expected, ISTA exhibits a slow, monotonic convergence characteristic of its $\mathcal{O}(1/k)$ rate. Conversely, thanks to Nesterov's acceleration, FISTA demonstrates a much faster convergence rate of $\mathcal{O}(1/k^2)$.

Question 4: Experimenting with ϵ

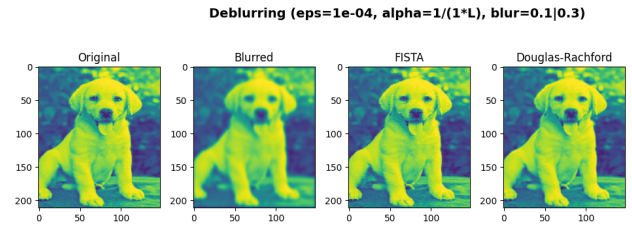
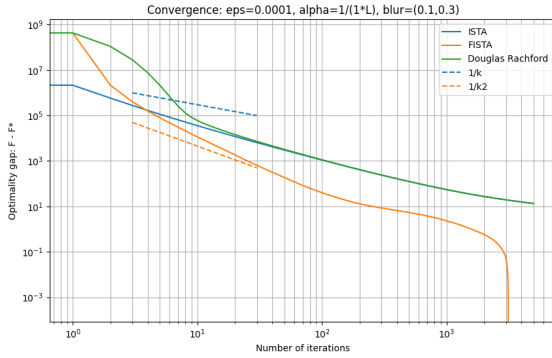


Figure 2: **Large Regularization** ($\epsilon = 10^{-4}$): The algorithm effectively removes noise by enforcing sparsity in the wavelet domain, resulting in a smooth reconstruction but slightly attenuating fine details.

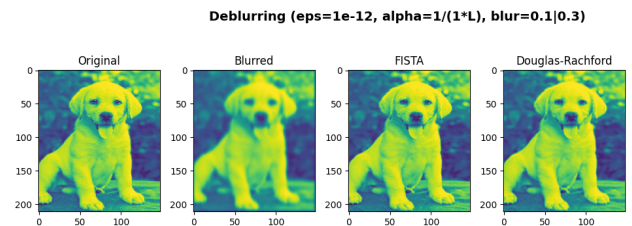
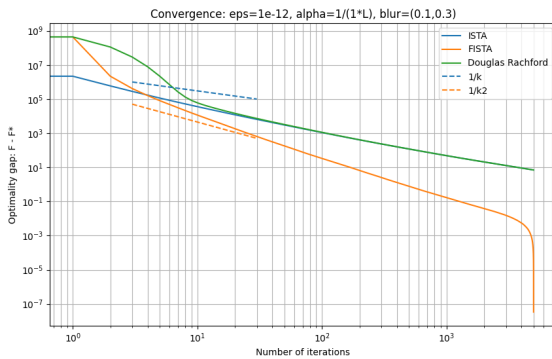


Figure 3: **Small Regularization** ($\epsilon = 10^{-12}$): Reducing the ℓ_1 penalty reduces the problem to an ill-conditioned least-squares inversion. This amplifies numerical noise and creates visible wavy artifacts in the reconstruction.

The parameter ε controls the strength of the ℓ_1 regularization term and therefore the sparsity of the solution. To interpret the role of ε , we rely on the ISTA solution formula established previously. Componentwise, the solution is given by the soft-thresholding operator

$$[x_{k+1}]_i = \text{sign}([v_k]_i) (|[v_k]_i| - \alpha\varepsilon)_+ \quad (8)$$

where α is the step size and can be for example $\alpha = \frac{1}{L}$ where L is the largest singular value of A . Hence, if $|[v_k]_i| \leq \alpha\varepsilon$, then $[x_{k+1}]_i = 0$: the effective threshold is $\alpha\varepsilon$. Increasing ε therefore increases the threshold, so more coefficients are set exactly to zero, small-amplitude variations in the pixels are removed, and fine details are suppressed, leading to a smoother reconstruction. At the same time, the stronger regularization improves conditioning and numerical stability, typically accelerating convergence. Conversely, decreasing ε lowers the threshold, fewer coefficients are annihilated, and more local variations (hence more details) are preserved; however, the problem then approaches the ill-conditioned least-squares inversion, making the reconstruction more sensitive to noise and generally increasing the number of iterations required for convergence.

Question 5: Experimenting with Step Sizes (α)

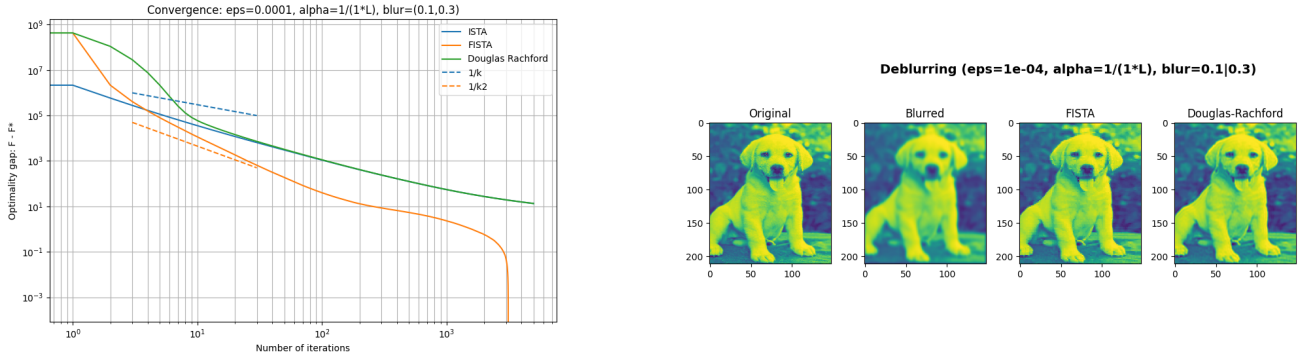


Figure 4: **Optimal Step Size ($p = 1$):** The step size is maximal ($\alpha = 1/L$). Convergence is extremely fast and smooth for both ISTA and FISTA, yielding a high-quality visual reconstruction rapidly.

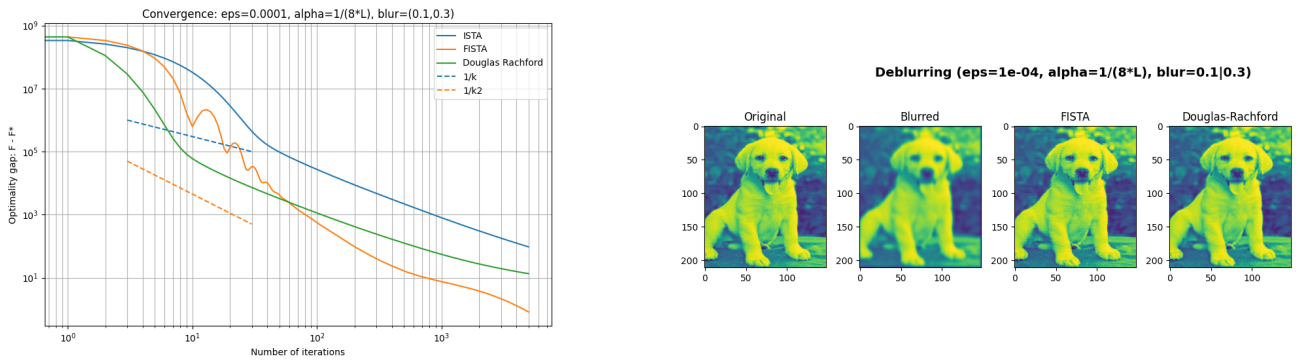


Figure 5: **Reduced Step Size ($p = 8$):** Severe “ripples” appear in FISTA’s convergence curve due to the momentum imbalance relative to the small gradient steps. Overall convergence is slowed down.

We investigated the effect of the step size by setting $\alpha = \frac{1}{pL}$ with $p \geq 1$, where L is the largest singular value of A . When $p = 1$, the step size is maximal and ensures the fastest theoretical convergence rate ($\mathcal{O}(1/k)$ for ISTA and $\mathcal{O}(1/k^2)$ for FISTA).

When $p > 1$, the step size α is reduced. Although the theoretical minimizer is unchanged as long as $\alpha \leq 1/L$, the convergence becomes slower.

For ISTA, reducing α simply decreases the magnitude of the gradient step at each iteration. The objective function remains monotonically decreasing, but more iterations are required to reach the same precision.

Therefore, for a fixed number of iterations, a smaller α results in a less accurate reconstruction and thus poorer visual quality.

For FISTA, the situation is more subtle. The update can be written in the form

$$x_{k+1} = \text{prox}_{\alpha\epsilon} \left(y_{k+1} + \gamma_k (y_{k+1} - y_k) \right),$$

where $y_{k+1} = x_k - \alpha \nabla f(x_k)$ represents the gradient correction, and $\gamma_k (y_{k+1} - y_k)$ is the extrapolation (inertial) term. The first term drives the iterate toward the minimizer through the gradient step, while the second one pushes it further in the previous direction.

When α is reduced, the gradient correction contained in y_{k+1} becomes weaker, since it is proportional to $\alpha \nabla f(x_k)$. In contrast, the inertial coefficient γ_k does not depend on α and continues to amplify the displacement $(y_{k+1} - y_k)$. As a result, the relative influence of the inertial term increases: the iterate may move too far past the minimizer before the gradient is able to compensate. The algorithm then corrects its trajectory, but the inertia may again push it in the opposite direction, producing the visible oscillations (“ripples”) observed in the objective curve.

The proximal operator

$$\text{prox}_{\alpha\epsilon}([v_k]_i) = [x_{k+1}]_i = \text{sign}([v_k]_i) (|[v_k]_i| - \alpha\epsilon)_+$$

acts as a contraction by shrinking the coordinates and possibly setting small ones to zero. It therefore introduces a damping effect, which slightly attenuates the oscillations. However, it does not remove the inertial mechanism itself, so the non-monotonic behavior persists. Despite these oscillations, the method eventually stabilizes and retains its accelerated convergence rate.

Question 6: Experimenting with Blurring Parameters

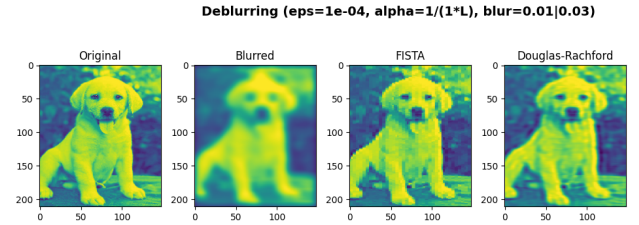
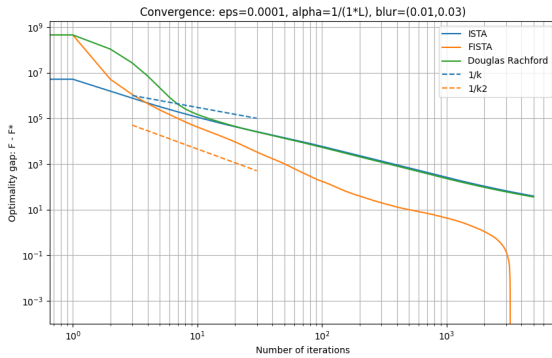


Figure 6: **Weak Blur** ($bz = 0.01, bx = 0.03$): The operator A is well-conditioned (close to the identity). Algorithms rapidly recover the original image with very high fidelity.

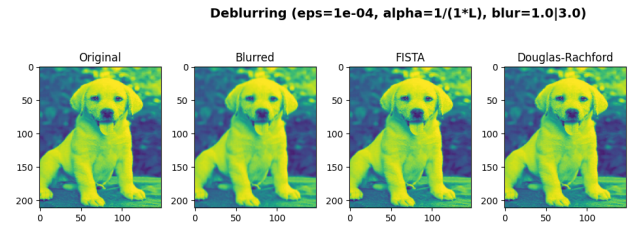
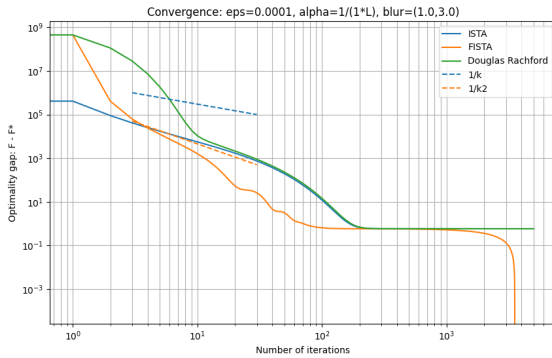


Figure 7: **Strong Blur** ($bz = 1.0, bx = 3.0$): The ill-conditioned operator A significantly slows down convergence in flat spectral directions. High-frequency details are permanently lost during the blurring process, leading to a softer visual reconstruction.

The parameters bz and bx control the variance of the Gaussian blur and therefore the conditioning of the operator A .

With weak blur ($bz = 1, bx = 3$), the operator A is close to the identity matrix. In that case, the singular values of A remain well distributed and bounded away from zero, so the problem is well-conditioned. The objective function

$$f(x) = \frac{1}{2} \|Ax - b\|^2$$

is well curved in all directions, which allows the algorithms to converge rapidly. Indeed, we can write $A^T A = V \Sigma^2 V^T$, so for each singular direction v_i ,

$$A^T A v_i = \sigma_i^2 v_i,$$

and the gradient $\nabla f(x) = A^T(Ax - b)$ scales that direction proportionally to σ_i^2 . When the singular values are not too small, the gradient provides effective corrections in every direction. Since only limited information is lost during the blurring process, the reconstruction is visually accurate.

With strong blur ($bz = 0.01, bx = 0.03$), the operator A strongly attenuates fine-scale variations in the image. Many singular values of A become very small, which makes $A^T A$ ill-conditioned. In the corresponding directions, the gradient corrections are weak (as they are proportional to σ_i^2), so the algorithms progress slowly along those components. More importantly, strong blur removes a significant part of the fine information from the image. Since the reconstruction seeks an image x such that Ax matches the blurred observation, any detail that has been almost suppressed by A cannot be reliably recovered. The degradation of visual quality under strong blur is therefore not only a numerical issue, but also an intrinsic loss of information.

Question 7: Impact of the Sampling Parameter

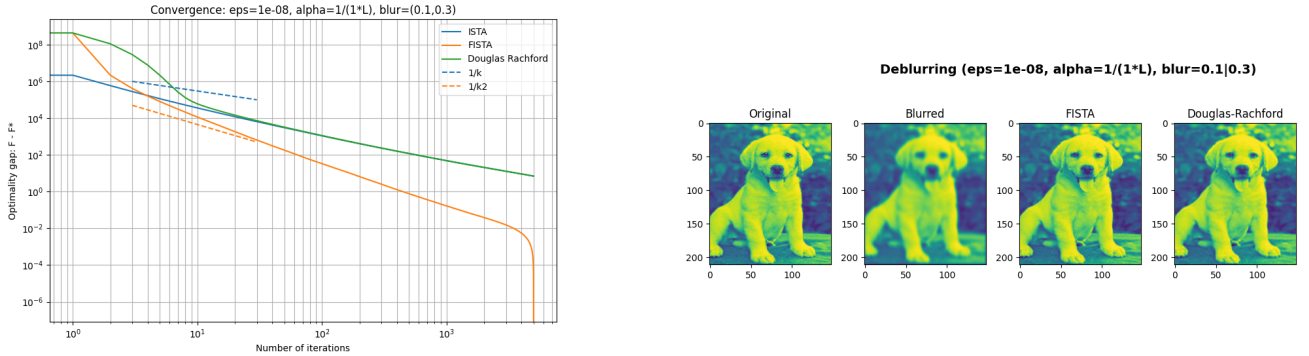


Figure 8: **Base Resolution (Sampling = 5)**: The problem dimension is relatively small. The algorithms compute each iteration very fast in terms of wall-clock time, but the inherent resolution of the reconstructed image is limited.

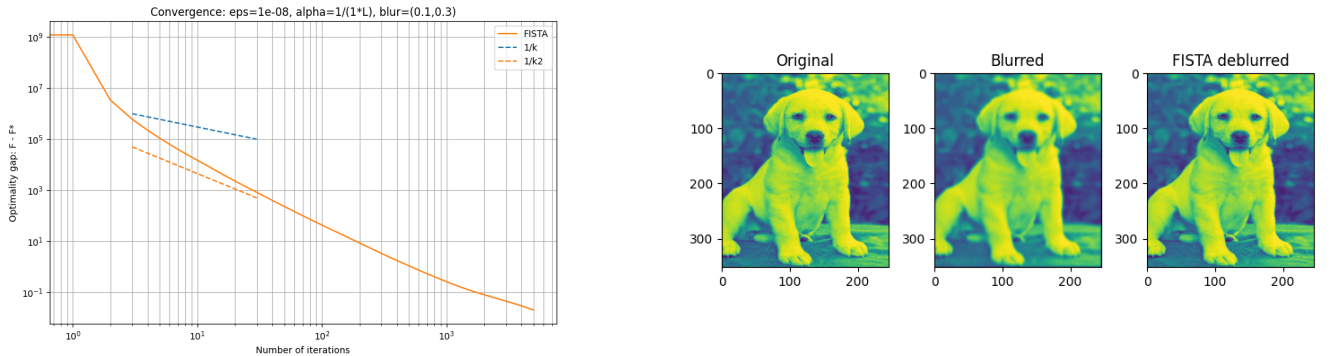


Figure 9: **Higher Resolution (Sampling = 3)**: The number of pixels is multiplied by approximately 2.77. While the algorithmic convergence behavior per iteration remains theoretically similar, the computational cost (wall-clock time) per iteration is drastically increased due to the larger 2D convolution and wavelet operators. However, the final visual reconstruction preserves much finer details.

Decreasing the sampling parameter (from 5 to 3 for the dog image) preserves a higher resolution of the original image. The total number of pixels n scales proportionally to $1/\text{sampling}^2$. Changing the sampling from 5 to 3 therefore multiplies the problem dimension by approximately $(5/3)^2 \approx 2.77$.

At each iteration, the dominant computational cost comes from large matrix–vector operations involving the operator A and its transpose. These operations require processing all image pixels, so their cost scales proportionally to the problem dimension n . Consequently, when the sampling parameter decreases and the image resolution increases, the number of pixels grows significantly, which directly increases the computational time per iteration.

While the theoretical convergence rates of the algorithms are unchanged, the increased problem size significantly raises the computational burden. This highlights the practical importance of accelerated schemes such as FISTA when processing higher-resolution images.

Question 8: Douglas-Rachford Splitting (Option I)

The Douglas-Rachford (DR) splitting algorithm aims to solve $\min_x f(x) + g(x)$. As stated in the assignment, we consider the formulation without an explicit step size. We assign the proximal-friendly ℓ_1 penalty to $g(x) = \epsilon\|x\|_1$, and the least-squares data fidelity term to $f(x) = \frac{1}{2}\|Ax - b\|_2^2$.

The DR iterations are defined as:

$$x_{k+1} = \text{prox}_g(z_k) \tag{9}$$

$$y_{k+1} = \text{prox}_f(2x_{k+1} - z_k) \tag{10}$$

$$z_{k+1} = z_k + y_{k+1} - x_{k+1} \tag{11}$$

The first step is simply the soft-thresholding operator evaluated at z_k with threshold ϵ . To compute the second step (Eq. 10), let $v = 2x_{k+1} - z_k$. The proximal operator of f evaluated at v is:

$$y_{k+1} = \arg \min_y \left(\frac{1}{2}\|Ay - b\|_2^2 + \frac{1}{2}\|y - v\|_2^2 \right). \tag{12}$$

By taking the gradient of this objective function with respect to y and setting it to zero, we obtain the optimality condition:

$$A^T(Ay - b) + (y - v) = 0. \tag{13}$$

Rearranging the terms to isolate y yields the following linear system:

$$(I + A^T A)y = v + A^T b. \tag{14}$$

Because the operator A involves large 2D convolutions and wavelet transforms, explicitly constructing and inverting the dense matrix $(I + A^T A)$ is computationally prohibitive. To circumvent this issue, we solve the linear system iteratively using the Conjugate Gradient (CG) method (`scipy.sparse.linalg.cg`) provided by the SciPy library. To ensure fast execution, we employ a "warm start" strategy, using the solution y from the previous iteration k as the initial guess (x_0) for the CG solver at iteration $k + 1$.

In practice, we observed that each Douglas–Rachford iteration is significantly more expensive than ISTA or FISTA, since it requires solving a linear system with Conjugate Gradient. However, the warm start strategy makes the inner CG solver converge in only a few iterations after the first steps.

Compared to FISTA, Douglas–Rachford appears more stable and less oscillatory in its behavior. Although it does not always reach the solution as quickly in terms of wall-clock time and iterations, its updates are often smoother and less sensitive to parameter tuning (there is not step size).